

Machine Learning Inference in the Programmable Data Plane

Raadhes Ch (CS22B069), Yashvardhan T(CS22B088), Shrijith B (CS22B093)
Group 12, CS6045 - SDN

May 20th, 2026

Abstract

This report presents our implementation of machine-learning inference inside a programmable data plane using P4, BMv2, and Mininet. The project focuses on PeerRush traffic classification, where packets are assigned to one of three peer-to-peer application classes: `emule`, `utorrent`, and `vuze`. We use a common packet-processing and feature-extraction pipeline, then map multiple model families onto switch-executable logic. The report evaluates a decision tree, a fixed-point linear classifier, and a compact binarized neural network (BNN), showing how model structure, arithmetic support, and approximate flow state affect switch-side inference. The code can be found on our project's github repository linked [here](#). The experiments can be run following the instructions in the readme file in the repository.

1 Introduction

Modern traffic-classification systems often run machine-learning inference on end hosts or backend servers after packets have already crossed the network. Programmable data planes make it possible to move some of this intelligence closer to the packets themselves. The goal of this project is to study how far simple ML inference can be pushed into a P4 switch pipeline while still preserving a realistic packet-processing workflow.

Our project follows a progressive implementation path inspired by prior work on in-network inference. We first studied existing systems such as IIsy, Pegasus, Planter, and N3IC, then selected a traffic-classification dataset and implemented switch-side models on BMv2. The completed experiments are a decision-tree classifier generated from a trained scikit-learn model, a fixed-point linear classifier with both BMv2-relaxed and real-switch-constrained P4 variants, and a BNN implementation that explores whether aggressively quantized neural inference remains useful when the switch must also parse packets and maintain flow features.

2 Prior Work

IIsy [1] demonstrates that traditional ML classifiers can be mapped into programmable-switch pipelines using match-action tables and hybrid switch/server classification. Its design is important for this project because it shows that small, interpretable classifiers can reduce server load and latency when they are placed directly in the network. However, IIsy is more naturally suited to models whose features are already available or can be supplied to the switch. Our work additionally considers flow features such as packet-length extrema and inter-packet delay, which require stateful register updates in the data plane.

Pegasus [2] targets scalable deep-learning inference on the data plane by decomposing neural-network computation into dataplane-oriented primitives: partitioning high-dimensional inputs, mapping lower-dimensional sub-vectors through table lookups, and reducing partial outputs. Pegasus motivated our broader project goal of moving beyond a single classifier type. For this report, however, we focus on smaller models that can be explained and evaluated within BMv2.

Planter [3] provides an automated model-to-P4 workflow for several ML models and targets. It has algorithms for Decision Trees, Random Forest, KNN, K Means, Bayes, Neural Networks and a few others. Its direct-mapping, encode-based, and lookup-based strategies are useful for understanding the tradeoff

between pipeline stages, table memory, and unsupported arithmetic. We initially intended to use Planter as a baseline, but the setup was not stable enough for a fair comparison in our environment. The repository depends on older library versions; installation failed with package build issues, and we had to roll Python back to 3.8 before the basic workflow started running. Even after that, BMv2 automation failed in the P4 VM because the scripts mixed environments and required Python modules that were not available under the sudo/P4 toolchain runtime. Planter is also not a direct baseline for the two model families we wanted to emphasize later in the project: it did not provide ready linear-classifier and BNN implementations matching our PeerRush flow-feature setting. For these reasons, we used Planter as design guidance rather than as a quantitative baseline.

N3IC [4] shows that binary neural networks can be compiled into NIC dataplanes using hardware-friendly operations such as XNOR and popcount. In N3IC, the switch-side program is mainly responsible for the binarized executor: it expects input features to have already been converted into binary feature vectors and supplied through a custom header. That design is elegant for a pure BNN datapath, but it moves an important part of the feature pipeline outside the normal packet parser. We could not identify a useful novelty in simply reproducing that custom-header approach. Instead, our BNN direction attempts a more traditional model over the real PeerRush features, where the switch extracts or reconstructs the input features itself and then performs quantized/binarized inference directly over those values. We try to show the difference in accuracy between the flexibility of BMv2 simulation and the constraints of real switch hardware.

3 Setting up Pegasus

While choosing this project topic the Pegasus paper, repository and youtube lectures were the first artifacts that we were introduced to (in this specific domain).

Pegasus is a framework for scalable deep learning inference on the data plane, published at SIGCOMM 2025 [2]. The reference implementation ¹ provides a lightweight BMv2 deployment of the MLP-B model. There are no other models on software provided by Pegasus.

While Pegasus is **not used as a baseline** we describe our efforts in understanding and running Pegasus.

3.1 Environment Setup

A Ubuntu 24.04 VirtualBox image suggested in the P4 tutorials ², specifically the build documented by Fingerhut ³ is used. The remaining Pegasus dependencies can be installed directly into this image via `pip` against the default virtual environment that comes up in the VM’s terminal (due to `bashrc`); the two environments are otherwise equivalent. A small number of Python version issues surfaced during dependency installation and were resolved by pinning compatible versions of the affected packages.

¹<https://github.com/afireswallow/Pegasus>

²<https://github.com/p4lang/tutorials>

³<https://github.com/jafingerhut/p4-guide/blob/master/bin/README-install-troubleshooting.md>

```

Pegasus > bmv2 >  requirements.txt
1 backcall==0.2.0
2 cffi==1.17.1
3 crcmod==1.7
4 decorator==5.2.1
5 getmac==0.9.5
6  grpcio==1.60.0
7 ipython==7.31.1
8 jedi==0.17.2
9 matplotlib-inline==0.1.7
10 -e git+https://github.com/minine
11 netifaces==0.11.0
12 numpy==1.4.2
13  numpy==1.26.4
14 p4runtime-shell
15  pandas==2.1.4
16 parso==0.7.1
17 pexpect==4.9.0
18 pickleshare==0.7.5
19 ply==3.11
20 prompt-toolkit==3.0.51
21 protobuf==3.20.3
22 psutil==7.0.0
23  pt
24 ptyprocess==0.7.0
25 pycparser==2.22
26 pygments==2.19.2

```

Figure 1: Python dependency issues and changes made.

3.2 Datasets

Three datasets are used in the original evaluation:

- **CICIOT2022** - IoT device traffic captures in Power, Idle, and Interact states, from the Canadian Institute for Cybersecurity ⁴.
- **ISCXVPN2016** - VPN-encrypted pcap traffic across seven application categories (Email, Chat, Streaming, FTP, VoIP, P2P), also from CIC/UNB ⁵.
- **PeerRush** - This dataset contains traffic generated by P2P applications, categorized into three classes (eMule, uTorrent, and Vuze)

For convenience, the preprocessed CSV splits used by the Pegasus authors are mirrored in the repository ⁶ and are sufficient for reproducing the BMv2 experiments. For the purpose of describing the process of running Pegasus we focus on CICIOT2022.

3.3 Pretrained Artifacts

The directory `bmv2/make_dataset/mlp/CICIOT2022/` contains the artifacts produced by `convert_mlp_pk1.py` for the CICIOT2022 dataset:

- `mlp.CICIOT2022_0.8563548831138729.pt` - the trained MLP-B model as a PyTorch `state_dict`, holding the weights and biases of the three fully-connected layers together with the batch-normalisation parameters ($\gamma, \beta, \mu, \sigma$). The numeric suffix is the held-out test accuracy of the checkpoint (85.6%).
- `mlp.CICIOT.pk1` - the table entries derived from the trained weights. This is the output of `convert_pk1_mlp.py`: fuzzy clustering thresholds, centroid-to-output mappings, and fused BN $\rightarrow$ FC $\rightarrow$ ReLU results, serialised for the control plane to install into the switch.
- `CICIOT2022.dataset.pk1` - the processed test set in per-packet form, suitable for replay through the switch via `listen_veth.py` and `tcpreplay`.

⁴<https://www.unb.ca/cic/datasets/iotdataset-2022.html>

⁵<https://www.unb.ca/cic/datasets/vpn.html>

⁶<https://github.com/afireswallow/Pegasus/tree/master/software/dataset>

3.4 Running the BMv2 Pipeline

The first step regenerates the table and dataset pickles from the checkpoint. Although this step is not strictly necessary when the distributed artifacts are used, it is a useful sanity check:

Listing 1: Convert csv to dataset pickle and create model pickle file.

```
cd bmv2/make_dataset

python convert_pickle_mlp.py \
  --model_path mlp/CICIOT2022/mlp_CICIOT2022_0.8563548831138729.pt \
  --test_data_path ../../dataset/CICIOT2022_test.csv \
  --dataset_type CICIOT2022 \
  --output_pickle_path mlp/CICIOT2022/output.pkl \
  --dataset_pickle_path mlp/CICIOT2022/dataset.pkl
```

The virtual ethernet pair used by BMv2 is brought up from the Pegasus/bmv2 directory:

```
bash start_veth.sh
```

Start the switch, then run the control plane in another terminal:

```
cd bmv2/
./p4_CICIOT2022/start_switch.sh # compile P4 and start BMv2

cd bmv2/p4_CICIOT2022/control/
python mlp32_control.py # populate match-action tables
```

Two things are flagged. First, the upstream invocation of the control plane is `python p4_CICIOT2022/control/mlp32_control.py` which is recommended in the repository's `bmv2/readme.md`.

Option 1: Simplified Startup (Recommended for Quick Test)

```
./p4_your_project/start_switch.sh # compile p4 and start bmv2
python3 p4_your_project/control/mlp32_control.py # start control plane
```

This option provides a convenient one-step startup: it compiles your P4 program, launches BMv2, and starts the control plane to push parameters.

But several paths inside the script are hard-coded relative to its own directory; running it from inside `p4_CICIOT2022/control/` (as `mlp32_control.py`) avoids the resulting `FileNotFoundError`s. Secondly, the path of the model pickle file was hardcoded which had to be changed in this python file.

```
13 current_dir = Path(__file__).parent
14 p4info_file_path = Path("../build/mlp_CICIOT.p4info.txt")
15 bmv2_json_path = Path("../build/mlp_CICIOT.json")
16 model_file_path = Path("/home/yang1210/THU_net/BMV2_Pegasus/make_dataset/mlp/CICIOT2022/mlp_CICIOT.pkl")
17
18 # p4c-bm2-ss --target bmv2 --arch v1model --p4runtime-files build/mlp_CICIOT.p4info.txt -o build/mlp_CICIOT.json basic.
19
```

Figure 2: Change made in `mlp32_control.py`

3.5 Traffic Replay and Classification

The directory `bmv2/send_receive/` contains helpers for replaying the test set through the switch. `gen_pcap.py` converts the dataset pickle into a pcap; it is hard-coded for CICIOT2022 but can be retargeted at the other

datasets without further changes, as all three share the same feature layout. Replaying then proceeds in two steps:

```
# You can listen to a port with this, type "1" when the port is prompted
sudo /home/p4/src/p4dev-python-venv/bin/python listen_veth.py

sudo tcpreplay --pps=100 -i veth0 ciclot_dataset.pcap # In a different terminal
```

Note that `sudo python listen_veth.py` fails because `sudo` does not inherit the activated virtual environment; the listener has to be launched with the venv interpreter directly, as above.

3.6 Debugging Aids

A few commands are useful when the switch fails to come up:

- `ps aux | grep simple_switch` - confirm that the `simple_switch` process is actually running.
- `sudo ss -tlnp | grep 50051` - confirm that the gRPC port (default 50051) is bound.
- `cat ~/Pegasus/bmv2/p4.CICLOT2022/bmv2logs/run_switch.log` - the switch log usually contains a precise error message when startup fails silently.

4 Dataset and Feature Pipeline

We use the PeerRush dataset for traffic classification. The labels used in this project correspond to three peer-to-peer applications:

- class 0: `emule`
- class 1: `utorrent`
- class 2: `vuze`

Each model uses the same ten-feature view of a packet or flow. Some features are extracted directly from parsed packet headers, while others are reconstructed as stateful per-flow metadata inside the P4 program.

Feature	Source	Meaning
<code>ip_len</code>	Header	IPv4 header length
<code>total_len</code>	Header	Total IPv4 packet length
<code>proto</code>	Header	Transport protocol number
<code>tos</code>	Header	IPv4 type-of-service field
<code>tcp_offset</code>	Header	TCP data offset
<code>max_packet_len</code>	Register state	Maximum packet length seen for the flow
<code>min_packet_len</code>	Register state	Minimum packet length seen for the flow
<code>max_ipd</code>	Register state	Maximum inter-packet delay for the flow
<code>min_ipd</code>	Register state	Minimum inter-packet delay for the flow
<code>last_ipd</code>	Register state	Inter-packet delay between the two latest packets

Table 1: PeerRush features used by the project models.

The stateful features are maintained using P4 registers. For each packet, the switch canonicalizes the flow key so that the two directions of a TCP flow map to the same state. A CRC-based index selects a register slot, and a tag or fingerprint is used to detect whether the slot likely belongs to the same flow. This design keeps the switch pipeline simple, but introduces the possibility of hash collisions and approximate flow state, though for this dataset we do not have enough flows that may cause collisions, but a switch over the internet may have flows ranging from a few thousands to millions at a time which may cause a lot of collisions and the model will likely fail for such cases. But it is still useful within a smaller subset of the internet.

Listing 2: Representative P4 state used for flow features.

```

register<bit<1>>(65536) valid0_reg;
register<bit<32>>(65536) tag0_reg;
register<bit<32>>(65536) last_ts0_reg;
register<bit<16>>(65536) max_len0_reg;
register<bit<16>>(65536) min_len0_reg;
register<bit<32>>(65536) max_ipd0_reg;
register<bit<32>>(65536) min_ipd0_reg;

```

Listing 3: Updating packet-length and inter-packet-delay state.

```

if ((meta.valid0 == 1w1) && (meta.stored_tag0 == meta.tag)) {
    meta.last_ipd = (meta.now_ts >= meta.stored_last_ts)
        ? (meta.now_ts - meta.stored_last_ts) : 32w0;
    if (meta.total_len > meta.max_packet_len) {
        meta.max_packet_len = meta.total_len;
    }
    if (meta.last_ipd > meta.max_ipd) {
        meta.max_ipd = meta.last_ipd;
    }
} else {
    meta.last_ipd = 32w0;
    meta.max_packet_len = meta.total_len;
    meta.min_packet_len = meta.total_len;
}

```

5 Implementation

5.1 Common P4 Pipeline

All submitted P4 programs target the `v1model` architecture used by BMv2’s `simple_switch`. They use the same packet-processing skeleton so that differences in results come from the model and state design, not from different test plumbing. The shared pipeline is:

1. Parse Ethernet, IPv4, and TCP headers.
2. Extract header-derived features into metadata.
3. Compute a canonical flow key and hash index.
4. Read and update per-flow registers for length and inter-packet-delay features.
5. Run the selected model’s inference logic.
6. Forward the packet to a class-specific output port.

The forwarding convention is fixed across all P4 experiments: class 0 is sent to port 1, class 1 to port 2, and class 2 to port 3. In the Mininet topology these ports connect to `h2`, `h3`, and `h4`, respectively. Therefore, the benchmark can infer the predicted class from the receiving host without adding custom prediction headers.

The Python code is used for training and model export. For the linear model, the final Python baseline is evaluated on the same `redeal.test.json` replay subset as the P4 benchmark, while maintaining the same packet-history features in software. This gives a cleaner comparison against the switch replay than evaluating only on precomputed CSV feature rows. The P4 variants use the same exported model; differences between constrained and BMv2-relaxed runs come from implementation details such as arithmetic support, register-bank size, fingerprint width, and collision handling.

5.2 Decision Tree

The decision-tree classifier is the strongest classical baseline in this report. We trained a scikit-learn decision tree on the PeerRush training data and limited the maximum depth to 5. This cap keeps the generated P4 code small enough to reason about and avoids creating an unrealistically deep chain of packet-processing decisions.

The trained model is exported to an intermediate representation and converted into P4 code. scikit-learn represents a tree using arrays for node features, thresholds, and child pointers. The converter traverses these arrays and emits nested P4 comparisons. The left branch represents a feature value less than or equal to the threshold, and the right branch represents a feature value greater than the threshold.

Because P4 operates on integer packet fields, floating-point thresholds from the Python model are converted into integer comparisons. For example, a Python-side comparison such as `max_packet_len <= 3082.5` becomes an integer comparison in P4. This is a necessary simplification, but it can contribute to accuracy loss compared with the original Python model.

The trained decision tree achieves **86.007%** test accuracy in Python. The exported tree has depth **5**, **63** total nodes, and **32** leaves.

5.3 Linear Classifier

The PeerRush task is a three-class classification problem, so the linear model is implemented as a scoring classifier: each class receives a weighted sum of the ten features, and the class with the largest score is selected. The Python training script, `linear_p4/scripts/train_linear.py`, trains the model from `PeerRush_train.csv` and evaluates it on `PeerRush_test.csv`. It standardizes the features, fits a regularized least-squares **one-vs-all classifier**, and then converts the learned coefficients back into raw-feature integer weights for P4.

The exported model is written to `linear_p4/generated/linear_model.json`. This file stores the feature order, label order, quantization scale, a 3x10 integer weight matrix, and three integer biases. The same quantized model is used by the Python baseline and by the BMv2-relaxed P4 program. The floating-point model is converted to fixed point by multiplying each raw coefficient by 8192 (2^{13}) and rounding to the nearest integer. At inference time, the switch computes only integer scores, and the common scale factor does not affect the final **argmax**. This reduces the learned coefficients from floating-point values to small signed integer weights such as -1123, 635, and 488, while biases are stored as signed integer offsets. The constrained P4 program implements the same scoring idea with hardware-conscious shift/add arithmetic:

Listing 4: Linear classifier scoring rule.

```
score[class] = bias[class]
for feature in 0..9:
    score[class] += weight[class][feature] * feature_value[feature]
prediction = argmax(score[0], score[1], score[2])
```

We implemented two linear P4 variants because BMv2 and hardware-conscious P4 expose different practical constraints. The relaxed BMv2 version, `linear_bmv2_relaxed.p4`, uses runtime-loaded registers for weights and biases and performs signed multiplication directly. This gives us considerable freedom during development: we can retrain the Python model, export a new JSON file, and reload the switch through the control plane without regenerating the P4 source.

Listing 5: BMv2-relaxed linear inference with runtime-loaded weights.

```
register<int<16>>(30) lin_w_reg;
register<int<32>>(3) lin_b_reg;

action mul_add(in int<16> w, in int<32> x, inout int<32> acc) {
    acc = acc + (((int<32>)w) * x);
}
```

The control-plane loader, `linear_p4/scripts/load_linear_model_cli.py`, reads the exported JSON and writes the 30 weights and three biases into BMv2 registers using `simple_switch_CLI`. It also installs

the class-to-port forwarding entries. This separation between P4 program and model parameters is useful for BMv2 experimentation because the data plane contains the scoring machinery, while the control plane supplies the current trained parameters.

Listing 6: Control-plane loading pattern for the relaxed linear model.

```
register_write lin_w_reg 0 <class_0_feature_0_weight>
...
register_write lin_w_reg 29 <class_2_feature_9_weight>
register_write lin_b_reg 0 <class_0_bias>
register_write lin_b_reg 1 <class_1_bias>
register_write lin_b_reg 2 <class_2_bias>
```

The constrained version, `linear_realswitch_constrained.p4`, is deliberately less permissive. It avoids multiplication, uses a single CRC16-indexed state bank, and hardcodes the trained coefficients into the P4 source as additions, subtractions, and shifts. This version is less flexible because changing the model requires regenerating or editing the P4 source, but it better reflects the arithmetic and state-access limitations one would expect on a real programmable switch. In the evaluation, the relaxed BMv2 program is treated as the functional upper bound for this model family, while the constrained program measures the cost of moving toward a more hardware-conscious implementation.

Listing 7: Constrained linear scoring using shift/add arithmetic.

```
meta.score0 = 32s7154;
meta.score0 = meta.score0 + meta.f1;
meta.score0 = meta.score0 + (meta.f3 << 6) + (meta.f3 << 5) + (meta.f3 << 2);
meta.score0 = meta.score0 - (meta.f4 << 10) - (meta.f4 << 6)
               - (meta.f4 << 5) - (meta.f4 << 1) - meta.f4;
```

5.4 Binarized Neural Network

The BNN design tests whether neural inference remains useful after aggressively quantizing both arithmetic and model structure for the data plane. Unlike N3IC-style designs that assume a custom binary feature header, our BNN parses normal Ethernet/IPv4/TCP packets and uses the same ten PeerRush features as the decision tree and linear models. The switch is therefore responsible for both flow-feature maintenance and model inference.

The deployable BNN architecture is a hybrid quantized network:

10 integer features -> 8 hidden neurons -> 3 class scores

The first layer uses ternary weights in $\{-1, 0, +1\}$, integer accumulators, and learned integer thresholds. Each hidden neuron computes a signed sum over the ten features and emits a binary activation. The output layer applies ternary weights to the eight hidden activations and selects the class with the largest score.

Listing 8: BNN hidden activation and class-score selection in P4.

```
acc[h] = bias[h]
for feature in 0..9:
    acc[h] += ternary_weight[h][feature] * feature_value[feature]
hidden[h] = (acc[h] > threshold[h])

score[class] = output_bias[class]
for h in 0..7:
    score[class] += output_weight[class][h] * hidden[h]
prediction = argmax(score[0], score[1], score[2])
```

The trained BNN parameters are stored in `bnn_p4/generated/model.json`. The P4 programs do not bake the learned weights into source code. Instead, `bnn_p4/scripts/load_model_cli.py` writes the first-layer weights, hidden thresholds, hidden biases, output weights, output biases, and class-forwarding table

entries into BMv2 through `simple_switch.CLI`. This keeps feature extraction and inference logic in P4 while allowing the model parameters to be changed at runtime.

We evaluate two BNN P4 implementations. Both load the same `model.json` parameters; only the switch-side implementation constraints differ:

- **Constrained BNN:** `peerrush_bnn_constrained.p4` uses one 4096-slot CRC16-indexed state bank and a 16-bit fingerprint tag. This intentionally creates visible state pressure, making it a proxy for tighter switch memory.
- **BMv2-relaxed BNN:** `peerrush_bnn_bmv2_relaxed.p4` uses two 65536-slot banks, 32-bit fingerprint tags, and older-slot overwrite. It is less constrained in state memory while keeping the same 10x8x3 BNN model.

6 Evaluation Methodology

6.1 General evaluation and model-wise evaluation

All measurements in this report use BMv2 and Mininet. The benchmark topology contains one switch and four hosts.

Host	Role	Meaning
h1	Sender	Replays PeerRush packets using Scapy
h2	Receiver	Class 0 sink
h3	Receiver	Class 1 sink
h4	Receiver	Class 2 sink

Table 2: BMv2/Mininet benchmark topology.

The benchmark script compiles the selected P4 program to BMv2 JSON, launches the Mininet topology, replays PeerRush records, and records which receiver obtains each packet. The same benchmark harness is used for the submitted BNN and linear P4 programs. The exact commands, expected runtimes, generated files, and expected outputs are documented in the repository `README.md`; the report focuses on the experimental setup and the interpretation of the results.

The sender script builds packets from `PeerRush/redeal_test.json`. The TCP sequence number is used only as a benchmark correlation identifier. Ground-truth labels and send timestamps remain in the sender-side summary files, while the classifier itself infers only from normal Ethernet/IPv4/TCP headers and maintained flow state. Receiver-side logs infer the predicted class from the host that received the packet: **h2** for class 0, **h3** for class 1, and **h4** for class 2.

For linear-model testing we also evaluated the benchmark script’s `--speedup` option. The replay dataset contains packet timestamps, and the sender normally sleeps according to the inter-packet gaps in those timestamps. With `--speedup 1000`, each positive inter-packet sleep is divided by 1000. This does not change packet contents, model weights, or switch inference logic; it only compresses the replay timeline. Since BMv2 and Mininet are software components, very high replay rates can still affect queueing and packet loss, so we compared speedup against the non-speedup run before using it for further experiments.

6.2 Latency Baseline

To isolate the per-packet cost of the in-network classifier from unrelated overheads in BMv2 (header parsing, table lookup, Mininet veth links), we introduce `simple_p4/p4/simple_forward.p4`, a minimal P4 pipeline that parses Ethernet/IPv4/TCP identically to the classifier programs but forwards purely by destination MAC via a small `const entries` match-action table. It contains no flow-state registers, no model evaluation, and no arithmetic beyond the table key match.

This baseline acts as a *perfect classifier*, in `packet_sender.py` we stamp the per class unicast MAC of the correct receiver host (**h2**, **h3**, or **h4**) on each outgoing packet. The classifier P4 programs do not key on

`hdr.ethernet.dstAddr` - they set `standard_metadata.egress_spec` from their own model output. Under `simple_forward.p4`, every packet is forwarded to the host matching its ground-truth class, yielding $\approx 100\%$ classification accuracy by construction. The latency difference between a classifier pipeline and the baseline therefore reflects *only* the model-evaluation cost in BMv2; the parser cost, table lookup, host-side scapy overhead, and veth traversal cancel out in the subtraction.

Pacing. The latency experiment is run *without* `--no-timing` and *without* `--speedup`, i.e. with the trace’s natural inter-packet timing. Replaying back-to-back (`--no-timing`) or at very high speedups can burst packets into BMv2 faster than the software switch can drain them, so measured latencies would absorb queueing delay inside the simulator rather than reflect pure per-packet processing cost. Natural pacing keeps inter-packet gaps wide enough for each packet to be fully processed before the next arrives.

Metric. For every packet, latency is the one-way, end-to-end interval $t_{\text{recv}} - t_{\text{send}}$, where t_{send} is captured on `h1` immediately before `scapy.sendp()` returns and t_{recv} is captured on the receiving host when `scapy.sniff()` delivers the packet to its callback. Sender and receiver records are matched by the TCP sequence number, which the sender re-uses as a per-packet identifier. The reported interval therefore covers `sendp` $\rightarrow$ `h1` veth $\rightarrow$ BMv2 ingress parser $\rightarrow$ P4 ingress pipeline $\rightarrow$ BMv2 egress queue $\rightarrow$ receiver veth $\rightarrow$ `sniff` callback. Dropped packets do not contribute. We report the mean as well as the p50, p95, and p99 percentiles to capture both typical and worst-case behaviour.

Workload and interpretation. All pipelines are replayed with the same `--count 30` prefix of `redeal_test.json` (652 packets at natural pacing) so that each classifier sees exactly the same input flows under identical sender conditions. We report each classifier’s mean latency alongside the `simple_p4` baseline; the difference, Δ_{mean} , is our headline indicator of per-packet classifier cost in BMv2. Since BMv2 is a software simulator, absolute latency values are representative of BMv2 on the P4 development VM rather than of any particular hardware target. The *relative* ordering of Δ_{mean} across pipelines is the portable signal: it reflects intrinsic differences in pipeline complexity (number of multiplications, register accesses, branches taken per packet) that carry over qualitatively to ASIC and FPGA targets.

6.3 Planter Decision Tree Comparison

We compare our decision tree against Planter’s decision-tree generator on the same workload. Both pipelines train a sklearn `DecisionTreeClassifier` with `max_depth=5` on `PeerRush_train.csv` and evaluate on the first 300 records of `redeal_test.json` (11,957 packet rows).

A important difference is feature availability: our pipeline supplies all ten PeerRush features, including the five stateful per-flow summaries; Planter is restricted to the five stateless header fields (`ip_len`, `total_len`, `proto`, `tos`, `tcp_offset`) because the generated P4 has no register-backed flow state.

For each pipeline we record the Python sklearn accuracy on the 11,957-packet test split, the on-switch BMv2 accuracy from a Mininet replay, and per-class recall.

7 Results

7.1 Decision Tree Results

Metric	Value
Python/sklearn test accuracy	86.007%
Tree depth	5
Total nodes	63
Leaves	32

Table 3: Decision-tree model characteristics before P4 deployment.

The BMv2 benchmark replayed 300 PeerRush records, which expanded into 11,957 packets. The observed switch-side results are:

Metric	Value
Packets sent	11,957
Packets received	11,717
Packets dropped	240
Drop rate	2.0072%
BMv2 classification accuracy	82.9807%
Aggregate throughput	53.94 Kbps
Mean latency	15.755 ms

Table 4: Decision-tree BMv2/Mininet benchmark results.

Class	Name	Accuracy
0	emule	73.7981%
1	utorrent	96.7475%
2	vuze	61.7712%

Table 5: Decision-tree per-class BMv2 accuracy.

The accuracy for the decision tree in Python and our implementation on P4 are very close, this is expected as the decision tree can be almost completely represented in the P4 code. The BMv2 accuracy is lower than the accuracy of the direct Python model. The following might be the reasons:

- Integer thresholds instead of floating point
- Hash collisions for flows which map to the same index

7.2 Linear Classifier Results

Before running packets through BMv2, we evaluated the exported linear model in Python on the same replay view used by the benchmark: the first 500 records of `redeal_test.json` with 1000x replay timing and software-maintained flow state. This is more comparable to the P4 run than the static CSV test rows, because the switch does not receive precomputed `max_packet_len`, `min_packet_len`, or IPD features; it reconstructs them during packet replay. On this replay baseline, the fixed-point exported linear model achieved 54.3476% accuracy. The floating-point reference achieved 54.2952%, and the two agreed on 99.3130% of packets, showing that weight quantization is not the dominant source of error.

The fixed-point conversion has a very small measured effect in the replay baseline: the fixed-point model is 0.0524 percentage points above the floating-point reference on this subset, which is within rounding and tie-breaking noise. On the original CSV test rows, the floating-point model achieved 76.1565% and the fixed-point export achieved 75.6716%, a loss of only 0.4849 percentage points. The much larger gap between Python replay and BMv2 P4 accuracy is therefore not mainly caused by coefficient quantization; it comes from replayed packet-state behavior, switch implementation details, and the constrained arithmetic/state design.

Metric	Value
Quantization scale	8192 (2^{13})
Replay records	500
Replay packets	19,068
Replay speedup	1000x
Fixed-point replay accuracy	54.3476%
Fixed-point replay macro-F1	49.5886%
Floating-point replay accuracy	54.2952%
Float/export agreement	99.3130%
CSV float accuracy	76.1565%
CSV fixed-point accuracy	75.6716%

Table 6: Python replay baseline for the exported linear model.

Class	Name	Support	Recall	Packets Predicted To Host
0	emule	7,226	33.0473%	3,337
1	utorrent	7,481	91.0974%	14,512
2	vuze	4,361	26.5994%	1,219

Table 7: Per-class Python replay baseline for the exported linear model.

```
(p4dev-python-venv) p4@p4dev:/media/sf_SDH/Project/linear_p4$ python scripts/train_linear.py --count 500 --speedup 1000
quant_scale: 8192
evaluation: redeal_json_state_replay
records: 500
speedup: 1000
accuracy: 0.543476
macro_f1: 0.495886
float_reference_accuracy: 0.542952
float_quant_agreement: 0.993130
csv_quantized_accuracy: 0.756716
host_accuracy_spread:
  h2/class 0 (emule): accuracy=0.330473, correct=2388/7226, predicted_to_host=3337
  h3/class 1 (utorrent): accuracy=0.910974, correct=6815/7481, predicted_to_host=14512
  h4/class 2 (vuze): accuracy=0.265994, correct=1160/4361, predicted_to_host=1219
(p4dev-python-venv) p4@p4dev:/media/sf_SDH/Project/linear_p4$
```

Figure 3: Python linear replay baseline on the first 500 `redeal_test.json` records with 1000x replay timing.

The relaxed BMv2 linear model was tested twice on 500 PeerRush records: once using the original replay timing and once using a 1000x replay speedup. The speedup reduced total benchmark runtime from 1750.254 seconds to 613.100 seconds, a reduction of about 65%. The aggregate accuracy changed only from 43.0302% to 42.9358%, a difference of 0.0944 percentage points. Based on this small aggregate accuracy change, we used the speedup setting for subsequent linear experiments to keep testing time manageable.

Metric	No speedup	Speedup 1000
Total runtime	1750.254 s	613.100 s
Packets sent	19,068	19,068
Packets received	18,663	18,665
Packets dropped	405	403
Drop rate	2.1240%	2.1135%
Classification accuracy	43.0302%	42.9358%
Aggregate throughput	49.16 Kbps	140.21 Kbps
Mean latency	22.871 ms	24.021 ms

Table 8: Effect of replay speedup on the relaxed BMv2 linear classifier.

Class	No speedup	Speedup 1000
0 / emule	39.8422%	32.8536%
1 / utorrent	51.0092%	60.8341%
2 / vuze	34.6251%	28.9383%

Table 9: Per-class accuracy (recalls) for the relaxed BMv2 linear classifier with and without replay speedup.

```
(p4dev-python-venv) p4@p4dev:/media/sf_SDN/Project$ ./run_benchmark.sh --count 500 linear_p4/p4/peerush_linear_two_bank_crc16_tagged.p4
[1/3] Cleaning any stale Mininet state
[2/3] Compiling peerush_linear_two_bank_crc16_tagged.p4 -> /media/sf_SDN/Project/build/classifier.json
[3/3] Starting BMv2 Mininet benchmark
Running command: python3 /media/sf_SDN/Project/test_scripts/mininet_benchmark.py --bm2-json /media/sf_SDN/Project/build/classifier.json --json-path /media/sf_SDN/Project/PeerRush/redeal_test.json --simple-switch-bin simple_switch --thrift-port 9090 --cli-bin simple_switch_CLI --speedup 1.0 --start-index 0 --max-frame-size 1500 --control-plane-type linear --control-plane-loader /media/sf_SDN/Project/linear_p4/scripts/load_linear_model_cli.py --model-path /media/sf_SDN/Project/linear_p4/generated/linear_model.json --count 500
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1
[benchmark] loading control plane: python3 /media/sf_SDN/Project/linear_p4/scripts/load_linear_model_cli.py --model /media/sf_SDN/Project/linear_p4/generated/linear_model.json --thrift-port 9090 --cli-bin simple_switch_CLI --bm2-json /media/sf_SDN/Project/build/classifier.json
[benchmark] sender started
[benchmark] [empty file] /tmp/h1-sender.log
[benchmark] [sender] replayed 100 records / 5088 packets in 399.6s
[benchmark] [sender] replayed 200 records / 9440 packets in 884.3s
[benchmark] [sender] replayed 300 records / 11901 packets in 1074.7s
[benchmark] [sender] replayed 400 records / 15681 packets in 1412.6s
[benchmark] }
=== PeerRush BMv2 Classification Benchmark ===
Total runtime: 1750.254 s
Total packets sent: 19068
Total packets received: 18663
Total packets dropped: 405
Drop rate: 2.1240%
Classification accuracy: 43.0302%
Aggregate throughput: 49.16 Kbps
Latency ms: mean=22.871 min=7.183 max=185.200 p50=21.281 p95=37.059 p99=55.655
Per-class accuracy:
class 0: 39.8422%
class 1: 51.0092%
class 2: 34.6251%
Per-receiver throughput and mean latency:
h2/class 0: throughput=17.42 Kbps mean_latency=24.465 ms packets=6936
h3/class 1: throughput=19.06 Kbps mean_latency=21.897 ms packets=8842
h4/class 2: throughput=12.69 Kbps mean_latency=21.995 ms packets=3663
*** Stopping 0 controllers
*** Stopping 4 links
...
*** Stopping 1 switches
s1 Terminated
(p4dev-python-venv) p4@p4dev:/media/sf_SDN/Project$
```

Figure 4: Relaxed BMv2 linear classifier without replay speedup.

```
(p4dev-python-venv) p4@p4dev:/media/sf_SDN/Project$ ./run_benchmark.sh --count 500 --speedup 1000 linear_p4/p4/peerush_linear_two_bank_crc16_tagged.p4
[1/3] Cleaning any stale Mininet state
[2/3] Compiling peerush_linear_two_bank_crc16_tagged.p4 -> /media/sf_SDN/Project/build/classifier.json
[3/3] Starting BMv2 Mininet benchmark
Running command: python3 /media/sf_SDN/Project/test_scripts/mininet_benchmark.py --bm2-json /media/sf_SDN/Project/build/classifier.json --json-path /media/sf_SDN/Project/PeerRush/redeal_test.json --simple-switch-bin simple_switch --thrift-port 9090 --cli-bin simple_switch_CLI --speedup 1000 --start-index 0 --max-frame-size 1500 --control-plane-type linear --control-plane-loader /media/sf_SDN/Project/linear_p4/scripts/load_linear_model_cli.py --model-path /media/sf_SDN/Project/linear_p4/generated/linear_model.json --count 500
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1
[benchmark] loading control plane: python3 /media/sf_SDN/Project/linear_p4/scripts/load_linear_model_cli.py --model /media/sf_SDN/Project/linear_p4/generated/linear_model.json --thrift-port 9090 --cli-bin simple_switch_CLI --bm2-json /media/sf_SDN/Project/build/classifier.json
[benchmark] sender started
[benchmark] [empty file] /tmp/h1-sender.log
[benchmark] [sender] replayed 100 records / 5088 packets in 152.2s
[benchmark] [sender] replayed 200 records / 9440 packets in 288.7s
[benchmark] [sender] replayed 300 records / 11901 packets in 360.3s
[benchmark] [sender] replayed 400 records / 15681 packets in 409.6s
[benchmark] }
=== PeerRush BMv2 Classification Benchmark ===
Total runtime: 613.100 s
Total packets sent: 19068
Total packets received: 18665
Total packets dropped: 403
Drop rate: 2.1135%
Classification accuracy: 42.9358%
Aggregate throughput: 148.21 Kbps
Latency ms: mean=24.021 min=7.771 max=224.603 p50=22.066 p95=40.168 p99=65.051
Per-class accuracy:
class 0: 32.8536%
class 1: 60.8341%
class 2: 28.9383%
Per-receiver throughput and mean latency:
h2/class 0: throughput=44.53 Kbps mean_latency=29.280 ms packets=5763
h3/class 1: throughput=59.47 Kbps mean_latency=21.050 ms packets=9442
h4/class 2: throughput=36.27 Kbps mean_latency=23.371 ms packets=3460
*** Stopping 0 controllers
*** Stopping 4 links
...
*** Stopping 1 switches
```

Figure 5: Relaxed BMv2 linear classifier with 1000x replay speedup.

We then compared the relaxed BMv2 linear program against the constrained, more hardware-conscious P4 program. The constrained version avoids signed multiplication and runtime-loaded weights, using shift/add logic instead. Under the 1000x speedup setting, the relaxed BMv2 program achieved 42.9358% accuracy, while the constrained program achieved 39.7944% accuracy. Compared with the 54.3476% Python replay baseline, the relaxed BMv2 program loses 11.4118 percentage points, and the constrained program loses 14.5532 percentage points. The constrained program is therefore 3.1414 percentage points below the relaxed BMv2 implementation in this run.

Metric	Relaxed BMv2	Constrained P4
Packets sent	19,068	19,068
Packets received	18,665	18,671
Packets dropped	403	397
Drop rate	2.1135%	2.0820%
Classification accuracy	42.9358%	39.7944%
Aggregate throughput	140.21 Kbps	151.07 Kbps
Mean latency	24.021 ms	21.262 ms

Table 10: Relaxed BMv2 linear classifier compared with the constrained P4 variant.

Class	Relaxed BMv2	Constrained P4
0 / emule	32.8536%	24.7025%
1 / utorrent	60.8341%	77.5698%
2 / vuze	28.9383%	0.0000%

Table 11: Per-class accuracy (recall) for relaxed and constrained linear P4 variants.

```
(p4dev:python-venv) p4@p4dev:~/media/sf_SDN/Project$ ./run_benchmark.sh --count 500 --speedup 1000 linear_p4/p4/peerush_linear_crc16_tagged.p4
[1/3] Cleaning any stale Mininet state
[2/3] Compiling peerush_linear_crc16_tagged.p4 -> /media/sf_SDN/Project/build/classifier.json
[3/3] Starting BMv2 Mininet benchmark
Running command: python3 /media/sf_SDN/Project/test_scripts/mininet_benchmark.py --bm2-json /media/sf_SDN/Project/build/classifier.json --json-path /media/sf_SDN/Project/PeerRush/redeal_test.json --simple-switch-bin simple_switch --thrif
t-port 9090 --cli-bin simple_switch_cli --speedup 1000 --start-index 0 --max-frame-size 1500 --count 500
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 1 switches
s1
[benchmark] sender started
[benchmark] [empty file] /tmp/h1-sender.log
[benchmark] [sender] replayed 100 records / 5000 packets in 130.2s
[benchmark] [sender] replayed 200 records / 9440 packets in 237.2s
[benchmark] [sender] replayed 300 records / 11981 packets in 337.6s
[benchmark] [sender] replayed 400 records / 15681 packets in 451.5s
[benchmark]
=== PeerRush BMv2 Classification Benchmark ===
Total runtime: 568.869 s
Total packets sent: 19068
Total packets received: 18671
Total packets dropped: 397
Drop rate: 2.0820%
Classification accuracy: 39.7944%
Aggregate throughput: 151.07 Kbps
Latency ms: mean=21.262 min=6.933 max=372.550 p50=19.603 p95=35.895 p99=56.251
Per-class accuracy:
class 0: 24.7025%
class 1: 77.5698%
class 2: 0.0000%
Per-receiver throughput and mean latency:
h2/class 0: throughput=33.16 Kbps mean_latency=22.759 ms packets=4228
h3/class 1: throughput=117.91 Kbps mean_latency=20.420 ms packets=14443
h4/class 2: throughput=0.00 bps mean_latency=0.000 ms packets=0
*** Stopping 0 controllers
*** Stopping 4 links
....
*** Stopping 1 switches
s1 Terminated
```

Figure 6: Constrained linear P4 result under 1000x replay speedup.

7.3 BNN Results

The BNN experiments use the same replay-style Python baseline as the linear model. `train_bnn.py` trains the deployable 10x8x3 quantized BNN from `PeerRush_train.csv`, exports `model.json`, and then evaluates

the exported integer model on the first 500 records of `redeal_test.json` with 1000x replay timing and software-maintained flow state. This is important because the earlier CSV-only BNN number used precomputed feature rows and the pre-rounded latent training parameters, while the deployed P4 path uses rounded integer weights, thresholds, and biases loaded from `model.json`.

Metric	Python BNN baseline
Replay records	500
Replay packets	19,068
Replay speedup	1000x
Accuracy	37.8960%
Macro-F1	18.3210%
Hidden units	8
Dominant prediction	class 0 / emule
CSV exported-model accuracy	27.8720%

Table 12: Python replay baseline for the exported 10x8x3 BNN model.

```
(p4dev-python-venv) p4app4dev:/media/sf_50M/Project$ cd bnn_p4/
(p4dev-python-venv) p4app4dev:/media/sf_50M/Project/bnn_p4$ python3 scripts/run_python_eval.py --model generated/model_bm2_relaxed.json --variant bm2_relaxed
variant: bm2_relaxed
packets: 232754
accuracy: 0.325429
macro f1: 0.163685
collisions: 0
evictions: 0
host_accuracy_spread:
h2/class 0 (emule): accuracy=1.000000, correct=75745/75745, predicted_to_host=232754
h3/class 1 (utorrent): accuracy=0.000000, correct=0/81764, predicted_to_host=0
h4/class 2 (vuze): accuracy=0.000000, correct=0/75243, predicted_to_host=0
(p4dev-python-venv) p4app4dev:/media/sf_50M/Project/bnn_p4$
```

Figure 7: Python BNN replay baseline on the first 500 `redeal_test.json` records with 1000x replay timing.

Class	Name	Support	Recall	Packets Predicted To Host
0	emule	7,226	100.0000%	19,068
1	utorrent	7,481	0.0000%	0
2	vuze	4,361	0.0000%	0

Table 13: Per-class Python replay baseline for the exported BNN model.

Metric	Constrained P4	BMv2-Relaxed P4
Packets sent	19068	19068
Packets received	18737	18747
Packets dropped	331	321
Drop rate	1.7359%	1.6834%
Classification accuracy	38.5620%	38.5410%
Aggregate throughput	181.81 Kbps	188.13 Kbps
Mean latency	23.098 ms	21.490 ms
p95 latency	32.257 ms	28.382 ms

Table 14: BNN BMv2 replay results for constrained and relaxed P4 programs.

Class	Constrained P4 Accuracy	BMv2-Relaxed P4 Accuracy
0 / emule	0.0000%	0.0000%
1 / utorrent	98.2390%	98.2355%
2 / vuze	0.0000%	0.0000%

Table 15: BNN per-class behavior. The deployed P4 variants send almost every delivered packet to class 1.

```

C:\p4dev-python-venv> p4dev\media\sf_SDN\Project$ ./run_benchmark.sh bnn_p4/p4/peerush_bnn_constrained.p4 --count 500 --speedup 1000
[1/3] Cleaning any stale Mininet state
[2/3] Compiling peerush_bnn_constrained.p4 -> /media/sf_SDN/Project/build/classifier.json
[3/3] Starting BMv2 Mininet benchmark
Running command: python3 /media/sf_SDN/Project/test_scripts/mininet_benchmark.py --bmv2-json /media/sf_SDN/Project/build/classifier.json --json-path /media/sf_SDN/Project/PeerRush/redeal_test.json --simple-switch-bin simple_switch --
thrif-port 9090 --cli-bin simple_switch_CLI --speedup 1000 --start-index 0 --max-frame-size 1500 --control-plane-type bnn --control-plane-loader /media/sf_SDN/Project/bnn_p4/scripts/load_model_cli.py --model-path /media/sf_SDN/Proje
ct/bnn_p4/generated/model.json --count 500
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
s1
*** Starting 1 switches
s1
[benchmark] loading control plane: python3 /media/sf_SDN/Project/bnn_p4/scripts/load_model_cli.py --model /media/sf_SDN/Project/bnn_p4/generated/model.json --thrif-port 9090 --cli-bin simple_switch_CLI --bmv2-json /media/sf_SDN/Proje
ct/build/classifier.json
[benchmark] sender started
[benchmark] [empty file] /tmp/h1-sender.log
[benchmark] [sender] replayed 100 records / 5080 packets in 131.9s
[benchmark] [sender] replayed 200 records / 9440 packets in 237.0s
[benchmark] [sender] replayed 300 records / 11901 packets in 296.3s
[benchmark] [sender] replayed 400 records / 15601 packets in 383.0s
[benchmark] }
=== PeerRush BMv2 Classification Benchmark ===
Total runtime: 473.809 s
Total packets sent: 19060
Total packets received: 18737
Total packets dropped: 321
Drop rate: 1.7355%
Classification accuracy: 30.5620%
Aggregate throughput: 181.81 Kbps
Latency ms: mean=23.098 min=9.479 max=127.659 p50=22.505 p95=32.257 p99=37.331
Per-class accuracy:
  class 0: 0.0000%
  class 1: 98.2090%
  class 2: 0.0000%
Per-receiver throughput and mean latency:
  h2/class 0: throughput=0.00 bps mean_latency=0.000 ms packets=0
  h3/class 1: throughput=181.93 Kbps mean_latency=23.098 ms packets=18737
  h4/class 2: throughput=0.00 bps mean_latency=0.000 ms packets=0
*** Stopping 0 controllers
*** Stopping 4 links
....
*** Stopping 1 switches
s1 Terminated

```

Figure 8: Constrained BNN P4 result under 1000x replay speedup.

```

Running command: python3 /media/sf_SDN/Project/test_scripts/mininet_benchmark.py --bmv2-json /media/sf_SDN/Project/build/classifier.json --json-path /media/sf_SDN/Project/PeerRush/redeal_test.json --simple-switch-bin simple_switch --
thrif-port 9090 --cli-bin simple_switch_CLI --speedup 1000 --start-index 0 --max-frame-size 1500 --control-plane-type bnn --control-plane-loader /media/sf_SDN/Project/bnn_p4/scripts/load_model_cli.py --model-path /media/sf_SDN/Proje
ct/bnn_p4/generated/model.json --count 500
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
s1
*** Starting 1 switches
s1
[benchmark] loading control plane: python3 /media/sf_SDN/Project/bnn_p4/scripts/load_model_cli.py --model /media/sf_SDN/Project/bnn_p4/generated/model.json --thrif-port 9090 --cli-bin simple_switch_CLI --bmv2-json /media/sf_SDN/Proje
ct/build/classifier.json
[benchmark] sender started
[benchmark] [empty file] /tmp/h1-sender.log
[benchmark] [sender] replayed 100 records / 5080 packets in 119.2s
[benchmark] [sender] replayed 200 records / 9440 packets in 219.9s
[benchmark] [sender] replayed 300 records / 11901 packets in 281.2s
[benchmark] [sender] replayed 400 records / 15601 packets in 367.4s
[benchmark] }
=== PeerRush BMv2 Classification Benchmark ===
Total runtime: 450.443 s
Total packets sent: 19060
Total packets received: 18747
Total packets dropped: 321
Drop rate: 1.6834%
Classification accuracy: 30.5410%
Aggregate throughput: 188.13 Kbps
Latency ms: mean=21.490 min=9.286 max=100.291 p50=21.158 p95=28.382 p99=32.549
Per-class accuracy:
  class 0: 0.0000%
  class 1: 98.2355%
  class 2: 0.0000%
Per-receiver throughput and mean latency:
  h2/class 0: throughput=0.00 bps mean_latency=0.000 ms packets=0
  h3/class 1: throughput=188.29 Kbps mean_latency=21.490 ms packets=18747
  h4/class 2: throughput=0.00 bps mean_latency=0.000 ms packets=0
*** Stopping 0 controllers
*** Stopping 4 links
....
*** Stopping 1 switches
s1 Terminated

```

Figure 9: BMv2-relaxed BNN P4 result under 1000x replay speedup.

The constrained and relaxed P4 results are extremely close: the relaxed version drops 10 fewer packets, improves throughput by 6.32 Kbps, and reduces mean latency by 1.608 ms, but its accuracy is lower by

only 0.021 percentage points. This means the larger relaxed state design does not solve the BNN accuracy problem. The limiting factor in this run is the learned/quantized decision surface and score selection, not flow-state collision pressure.

The more important inference is the output collapse. The replay-style Python baseline for the exported model collapses to class 0, while the recorded P4 run collapses to class 1. The older CSV/latent Python baseline collapsed to class 2. These different dominant classes are not contradictory model successes; they show that this BNN is extremely sensitive to the exact evaluation path: CSV rows versus replayed packet state, pre-rounded latent parameters versus exported integer parameters, and Python replay timing versus BMv2 runtime state. In all cases, the useful conclusion is the same: the current BNN does not preserve enough class separation for PeerRush. This is still a useful negative result because it explains why N3IC-style BNN datapaths rely on carefully encoded binary features, while our more traditional direct-feature BNN has to absorb feature extraction, scaling, thresholding, and classification inside a much tighter P4 pipeline.

7.4 Latency Baseline Comparison

This benchmarking here has been done a different system compared to the above results so there may be some deviation in absolute values. However, the essence of comparing the different models remains.

We replay the first 30 records of `redeal_test.json` (652 packets) through each pipeline under natural trace pacing, in line with the methodology of Section 6.2. Table 16 reports per-packet one-way latency statistics and drop rate. Δ_{mean} is the increase in mean latency over the `simple_forward` baseline and represents the per-packet classifier cost in BMv2.

Pipeline	Mean (ms)	Min (ms)	Max (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Δ_{mean} (ms)	Drop (%)
simple_forward (baseline)	19.915	3.349	276.188	12.362	71.330	103.374	—	4.29
linear_bmv2_relaxed	30.762	5.577	1408.229	16.483	87.063	184.930	+10.847	4.14
linear_realswitch_constrained	21.318	5.634	568.899	13.738	67.752	104.834	+1.403	2.45
generated_tree	27.136	4.126	2017.625	14.607	76.186	134.932	+7.221	2.91
peerrush_bnn_bmv2_relaxed	20.982	6.880	267.298	15.067	60.602	96.925	+1.067	1.84
peerrush_bnn_constrained	24.479	7.833	223.404	17.726	71.329	122.615	+4.564	3.83

Table 16: Per-packet end-to-end latency under natural trace pacing (30 records, 652 packets).

The multiplication-heavy `linear_bmv2_relaxed` pipeline is the most expensive at +10.85 ms over baseline, reflecting the per-feature `int<16> × int<32>` multiplies that BMv2 executes in software.

The `linear_realswitch_constrained` variant, which decomposes the same trained model into shift-and-add chains baked into the P4 source, collapses to only +1.40 ms. This confirms two things at once: the integer multiply is the dominant per-packet cost in the relaxed linear pipeline, and the hardware-targeted decomposition preserves the model’s logical behavior while removing essentially all of that cost.

The BNN relaxed variant achieves lower classifier cost at +1.07 ms, than the constrained BNN is heavier (+4.56 ms). The former uses 65K size flow state banks and larger hash widths.

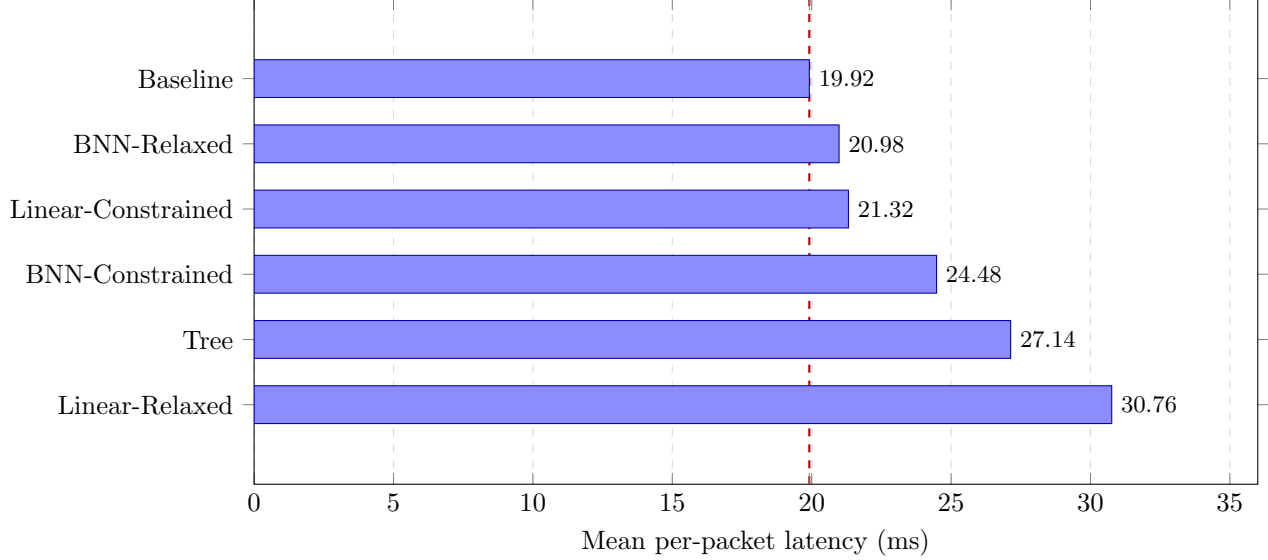


Figure 10: Mean per-packet end-to-end latency by pipeline. The top row (**Baseline**) is `simple_forward.p4` (no classifier); the dashed red vertical line marks its mean. Horizontal distance from that line to each other bar is the per-packet classifier cost (Δ_{mean}). Measured over 30 records (652 packets) at natural trace pacing.

We deliberately avoid `--no-timing`. To validate the methodological choice in Section 6.2, we re-ran the same baseline `simple_forward.p4` pipeline with the sender configured to ignore replay timestamps (`--no-timing`, i.e. scapy sends back-to-back as fast as possible):

Run	Mean (ms)	Min (ms)	Max (ms)	p50 (ms)	p95 (ms)	p99 (ms)	Drop (%)
simple_forward, natural pacing	19.915	3.349	276.188	12.362	71.330	103.374	4.29
simple_forward, <code>--no-timing</code>	29.189	4.417	211.097	16.679	96.051	163.546	3.37

Table 17: Effect of `--no-timing` on the baseline pipeline.

Even though `simple_forward` is the lightest possible pipeline and the packets themselves are unchanged, mean latency rises from 19.92 ms to 29.19 ms (+9.27 ms), the median rises from 12.36 ms to 16.68 ms, and the p95 rises from 71.33 ms to 96.05 ms. This added latency is entirely queueing delay inside BMv2: with back-to-back arrivals, packets sit waiting for the software pipeline to free up before they can be processed. Crucially, this queueing cost is *larger* than the per-packet classifier cost of the constrained linear or relaxed BNN pipelines in Table 16. If we had run the comparison with `--no-timing`, the queueing contribution would have a large effect on the latencies.

7.5 Comparison with Planter Decision Tree

The experimental setup and the rationale for restricting Planter to the five header-only features are described in Section 6.3. Table 18 reports the resulting accuracies on the 300 record and 11,957-packet test split used for the decision tree evaluation.

	Features	Python acc.	BMv2 acc.	Class 0	Class 1	Class 2
Our tree_p4	10	86.01%	82.98%	73.80%	96.75%	61.77%
Planter DT	5	76.04%	76.26%	67.01%	88.12%	68.52%

Table 18: Decision tree comparison on the same 11,957-packet test split. Per-class columns report recall (true-class accuracy).

A few findings:

The ten extra percentage points of Python accuracy come from the stateful features. Adding `max_packet_len`, `min_packet_len`, and the three IPD statistics raises the offline sklearn accuracy from 76.04% to 86.01%. The decision tree splits heavily on these IPD statistics: P2P applications have distinguishable timing signatures that header-only features cannot capture.

Planter’s table generation is essentially lossless while our implementation loses 3%. Planter’s BMv2 accuracy (76.26%) is within rounding of its Python sklearn accuracy (76.04%) - the generated ternary match-action entries exactly encode the tree’s behaviour. Our BMv2 accuracy (82.98%) drops about three percentage points below the offline Python accuracy (86.01%) likely because the stateful features themselves are imperfect on the switch: per-flow registers are hash-indexed with collisions, and IPD measurements are quantised to the switch’s timestamp resolution. The Python replay simulates these approximations to the extent the offline replay can, but real BMv2 state behaviour adds a small additional gap. So our tree retains the stateful-feature advantage but pays for it in deployment-time accuracy loss; Planter has no such gap because it has no such state.

Class-balance shifts. Our pipeline’s per-class behaviour is strongly biased toward class 1 (**utorrent**) - 96.75% recall vs 73.80% and 61.77% for the other two classes. Planter’s class distribution is more uniform: class 1 still leads at 88.12%, but class 2 (**vuze**) recovers to 68.52% versus our 61.77%. The stateful features clearly help **utorrent**’s separability the most on PeerRush but disadvantage **vuze**.

In summary, on the same data, the same tree depth, and the same target (BMv2), our pipeline produces a ~ 7 -percentage-point higher deployment accuracy than Planter’s decision tree because we extract and use stateful per-flow features that Planter’s compiler cannot generate P4 for. Planter, in turn, demonstrates a cleanly lossless model-to-table compilation pipeline that our hand-rolled approach does not need but also does not provide.

8 More Discussion

The decision-tree result shows that a compact classical ML model can be translated into P4 and evaluated as part of a realistic packet path. The Python model accuracy is 86.007%, while the BMv2 replay accuracy is 82.9807%. This drop is expected because the switch version uses integer comparisons and approximate per-flow state. The P4 implementation also depends on register indexing, so hash collisions can cause a packet to inherit stale or unrelated flow statistics.

The per-class results show that class 1, **utorrent**, is classified much more accurately than classes 0 and 2 in the measured decision-tree run. The linear classifier shows a similar class-1 bias even before P4 deployment: on the Python replay baseline it predicts 14,512 of 19,068 packets as class 1, giving high **utorrent** recall but weaker **emule** and **vuze** recall. BMv2 then adds further loss from packet replay, state approximation, register-index behavior, and implementation constraints. The BNN is more extreme still, because the deployed P4 runs send effectively all delivered packets to the class-1 receiver. These results show that aggregate accuracy alone is not enough for this task; per-class behavior is essential for understanding whether a model is truly separating PeerRush applications or just exploiting class imbalance.

The linear and BNN implementations represent two different directions for data-plane ML. The linear classifier is simple and compact, but arithmetic support matters: direct multiplication is convenient in BMv2, while the constrained P4 version must approximate scoring with shifts and additions. The BNN is more

aligned with bit-level data-plane operations, but the PeerRush experiment shows that this alignment is not enough by itself. When the model uses ternary weights, binary hidden activations, integer thresholds, and switch-maintained flow features, the current BNN loses class separation and collapses to a dominant output. This explains why a custom binary-feature pipeline, as in N3IC, can be attractive even though we deliberately avoided that assumption in order to keep the switch responsible for parsing and feature extraction.

9 Hardware Feasibility and Limitations

The decision tree is the most mature implementation in this report and is relatively switch-friendly. A depth-5 tree with 63 nodes and 32 leaves uses integer comparisons, assignments, hashes, and register reads/writes. These operations are closer to what programmable switches can support than full-precision neural-network arithmetic.

The main limitation is state. Features such as maximum packet length and inter-packet delay are not available in a single packet header, so the switch must maintain approximate per-flow state. A CRC-indexed register design is practical for BMv2 experiments, but any real hardware discussion must consider memory footprint, collision behavior, register access limits, pipeline stage limits, and action complexity.

The linear and BNN experiments expose two additional hardware issues. First, arithmetic that is easy in BMv2 is not necessarily a realistic switch target, so the constrained linear implementation trades model flexibility for shift/add scoring. Second, BNNs reduce multiplication but still require many register reads, threshold comparisons, and score updates. The relaxed BNN state bank improves packet loss and latency slightly, but the accuracy remains essentially unchanged, indicating that memory scale alone is not enough when the quantized model itself is poorly separated.

BMv2 results should be interpreted as functional validation, not as line-rate hardware performance. BMv2 is a software switch, so its latency and throughput do not directly represent Tofino or SmartNIC behavior. Still, it is useful for validating parser logic, feature extraction, generated inference code, and end-to-end benchmark tooling.

10 Use of AI Assistance

We used AI assistance as an engineering and writing aid during this project. The AI did not choose the project goal independently, did not provide the dataset, and did not run experiments outside our project environment. The design choices, debugging decisions, experiment selection, benchmark runs, screenshots, and report were created and performed by the team, but parts were streamlined by AI companions. Some AI assistance includes Generating a report paragraph from grammatically incorrect notes and Assistance in generating simple code and syntactic help.

The primary LLM models used are Claude Opus 4. and ChatGPT.

10.1 Human Inputs and Contributions

Design choices, coding and modifications, intricate debugging, experiment selection, model selection, benchmarking, report design and content has been done by the team. (AI has been used for streamlining the process)

We supplied the project specification, selected PeerRush as the dataset, and defined the main research direction: compare multiple traffic-classification models inside a programmable data plane. The team also decided several important constraints that shaped the implementation:

- Use normal Ethernet/IPv4/TCP packets rather than custom feature or prediction headers.
- Infer packet features from headers and maintain flow statistics inside the switch.
- Use `h1` as the traffic source and infer predictions from the receiving host: `h2` for class 0, `h3` for class 1, and `h4` for class 2.
- Compare BMv2-relaxed and more hardware-conscious constrained P4 implementations without changing the underlying model.

- Keep the final repository minimal and use `run_benchmark.sh` as the central benchmark entry point.

We also performed the VM-side execution work: installing and checking dependencies such as Scapy, running BMv2/Mininet benchmarks, collecting logs, identifying runtime failures such as Thrift connection errors and host Python environment issues, adding result screenshots, and deciding which results were meaningful enough to report.

10.2 AI-Assisted Work

AI assistance was used to accelerate implementation, debugging, and documentation. The main prompts and requests included:

- Give a simple script to train a decision tree model.
- Give a simple script to train a BNN model.
- Give a simple script to train a linear classifier.
- Debug the test harness so packets are sent from `h1`, received on `h2/h3/h4`, and classified by destination host.
- Debug P4 compiler errors such as signed/unsigned and unsized literal issues in hidden activation assignments.
- Update the report tables with the new benchmark and Python-baseline results which were collected.
- What are the constraints on a Tofino switch?
- Generate a latex table and graph for the mean latencies of the various p4 switch pipelines. [Pastes the logs of benchmark runs]
- Are the current citations factually correct ?
- Based on our previous p4 programs [attach tree and bnn p4 code] create a simple forwarding program for packets.
- What is the issue with running the Planter.py code. [Paste seaborn terminal issue] (Solution raised to change seaborn to seaborn-v0.8)

The AI helped write and refactor Python scripts. All numerical results reported in this document come from scripts, benchmark outputs, generated metrics, or screenshots produced in the project environment. AI assistance was used to organize the results.

All AI-generated code was reviewed and tested by the team. Python scripts were checked against expectations and shell scripts were reviewed manually. Several AI suggestions were caught as incorrect during review. All numbers in the report and README come from benchmarking in the VM environment and no results are fabricated.

11 Contributions, Novelties & Work

- Understanding **existing work** - mainly Pegasus, Planter and IIsy.
- The choice of **dataset** choice, creating the data pipeline and implementing stateful features (though the derived features are motivated).
- Python **training scripts** for models and subsequent moving **model implementation** for Decision trees, Linear classifiers and BNNs to P4.
- Constraint variants of the linear classifier and BNN models.

- **Modification** and debugging of Pegasus and Planter repositories to work on our virtual machines.
- Reproducible benchmarking with provided scripts and instructions via the markdown files. Benchmarking across:
 - Comparing accuracy and other classification metrics against the **original python models**.
 - **Decision tree** compared with **Planter**’s implementation.
 - **Latency benchmarking** across models and comparing against a simple-forwarding switch

12 Conclusion

This project demonstrates a practical workflow for implementing ML inference in the programmable data plane. We studied prior systems, selected PeerRush traffic classification as the target problem, built a shared feature pipeline, and implemented decision-tree, linear, and BNN classifiers in P4. The decision tree is the strongest completed model, achieving 82.9807% BMv2 classification accuracy on a replay of 300 PeerRush records. The linear classifier is less accurate but useful for studying fixed-point scoring and the gap between BMv2-relaxed arithmetic and a more constrained switch-oriented implementation. The BNN result is a negative but informative outcome: although the P4 programs run end to end, the current quantized model collapses to one dominant output class, showing that the BNN implementation still needs work.

References

- [1] C. Zheng, Z. Xiong, T. T. Bui, S. Kaupmees, R. Bensoussane, A. Bernabeu, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, “IIsy: Practical In-Network Classification,” arXiv:2205.08243, 2022.
- [2] Y. Zhang, S. Yao, Y. Feng, K. Chen, T. Li, Z. Liu, Y. Zhao, L. Zhang, X. Gao, F. Xiong, Q. Li, and K. Xu, “Pegasus: A Universal Framework for Scalable Deep Learning Inference on the Dataplane,” ACM SIGCOMM, 2025.
- [3] C. Zheng, M. Zang, X. Hong, L. Perreault, R. Bensoussane, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman, “Planter: Rapid Prototyping of In-Network Machine Learning Inference,” *ACM SIGCOMM Computer Communication Review*, vol. 54, no. 1, 2024. DOI: [10.1145/3687230.3687232](https://doi.org/10.1145/3687230.3687232).
- [4] G. Siracusano, S. Galea, D. Sanvito, M. Malekzadeh, G. Antichi, P. Costa, H. Haddadi, and R. Bifulco, “Re-architecting Traffic Analysis with Neural Network Interface Cards,” USENIX NSDI, 2022.